SmartSolar Dashboard — `app.py` Summary

> **Presentation slide reference** — workflow and how it works

What It Is

A **live, interactive web dashboard** built with Streamlit that lets users predict real-world solar panel power output and battery backup time — powered by the 5 ML models trained in `solar_pipeline.py`.

How It Works — Step by Step

Step	What Happens
① Load	On startup, loads all trained models (`models/*.pkl`) and the scaler into memory once (cached — no re-loading on every interaction)
② User Input	User sets panel specs, weather conditions, real-world loss factors (soiling, aging, wiring, shading), and system config (battery, inverter, load) via the sidebar
③ Compute T_mod	NOCT formula calculates actual panel surface temperature from ambient temp + irradiance
④ ML Prediction	All 5 models run simultaneously on the 12-feature input → each returns a predicted DC power (Watts)
⑤ Ensemble	The 5 predictions are averaged into one ensemble output (`ens_pdc`)
⑥ Real-World Adjustment	The ensemble output is scaled down by user-specified loss factors: `adjusted = ens_pdc × (100 − losses%) / 100`
⑦ Energy & Backup	From adjusted DC watts → AC watts → daily energy (Wh) → battery backup hours
⑧ Display	Results shown as metric cards, output gauge, temperature display, backup chart, and 5 analysis tabs

What the User Sees

```
Sidebar Inputs
  └─ Panel specs · Weather · Loss factors · Battery / Load

Live Prediction Row
  └─ Per-model Watts · Ensemble W · Adjusted W · Backup hours

3 Visual Widgets
  └─ Output Gauge  |  Module Temperature  |  Backup Energy Chart

5 Analysis Tabs
```

```
   ├─ Hourly Profile    — how output changes hour-by-hour today
   ├─ Irradiance Sweep  — output vs. solar radiation (0–1100 W/m²)
   ├─ Temperature Sweep — thermal derating effect (0–50°C)
   ├─ Formula vs ML     — why ML outperforms the physics formula alone
   └─ Model Performance — R², RMSE, MAPE for all 5 models
```

Key Design Points

- **Physics formula shown as a baseline** — the ML models correct its deviations caused by real-world losses such as soiling, aging, shading, and wiring
- **No re-training inside the app** — `app.py` only performs inference; all learning happened in `solar_pipeline.py`
- **Any panel type supported** — user enters their own datasheet values or picks from 11 real-world presets (Jinko Solar, LonGi, SunPower, Trina, etc.)
- **Instant results** — predictions complete in milliseconds since models are pre-trained and cached with `@st.cache_resource`
- **Real-world loss control** — users adjust soiling, aging, wiring, and shading sliders to reflect their actual site conditions, giving a more accurate adjusted output than the raw ML prediction alone

---

Data Flow Diagram

```
solar_pipeline.py
  └─ trains models → saves models/*.pkl + scaler.pkl
        │
        ▼
app.py  (Streamlit dashboard)
  ├─ Sidebar  ──────────────────────────┐
  │   Panel specs                       │
  │   Weather inputs                    │
  │   Real-world loss sliders           │
  │   System config (battery, load)     │
  └─────────────────────────────────────┘
        │
        ▼
  NOCT formula → T_mod
  Physics formula → P_ideal  (baseline)
        │
        ▼
  predict_all_models()
  scaler.transform([12 features])
  → 5 models predict simultaneously
        │
        ▼
  Ensemble average → ens_pdc
        │
        ▼
  × combined loss factor → adjusted_pdc
        │
```

```
        ▼
  compute_backup()
  P_ac · E_panel · E_batt · backup_hrs
        │
        ▼
  Dashboard Output
  ├─ Metric cards (per model + ensemble + adjusted + backup)
  ├─ Loss breakdown expander
  ├─ Output gauge (adjusted vs rated Wp)
  ├─ Module temperature display
  ├─ Backup energy bar chart
  └─ 5 Analysis tabs (hourly · irradiance · temp · formula vs ML · model
performance)
```

Generated from `app.py` — SolarCast Capstone Project